

Arduino for Embedded Systems - Beginner Level

Lesson 4: Analog Input: Sensors and ADC

Analog Input: Sensors and ADC – Study Notes

Key Concepts

1. **Analog to Digital Converter (ADC)** – hardware that samples a continuous voltage and converts it to a discrete digital value (usually 0–1023 on a 10-bit Arduino ADC).
2. **Potentiometer as a voltage divider** – a three-terminal variable resistor that provides a tunable voltage between 0 V and the supply voltage (5 V on most Arduino boards).
3. **`analogRead()` function** – reads the voltage on an analog pin and returns the corresponding ADC count.
4. **Mapping values** – using `map()` (or arithmetic) to translate raw ADC counts into useful units (e.g., degrees, percentage, sensor-specific ranges).
5. **Serial Monitor** – a simple way to display data from the microcontroller on a computer for debugging and logging.

Summary

An analog input on an Arduino (or similar microcontroller) is read through its built-in ADC. The ADC samples the voltage present on an analog pin, converts that voltage into a 10-bit integer (0–1023 for a 0–5 V range), and makes the result available to your sketch via `analogRead()`.

A common beginner sensor is the **potentiometer**. By wiring its two outer leads to 5 V and GND and its middle wiper to an analog pin, the wiper voltage varies linearly with the knob position. This varying voltage can be read, scaled, and displayed on the Serial Monitor, giving you immediate visual feedback and a foundation for working with any analog sensor.

The typical workflow is:

1. **Wire the sensor** (potentiometer) correctly.
2. **Read the raw ADC value** with `analogRead()`.
3. **Map or scale** the raw value to a human-readable range (e.g., 0–100 %).
4. **Print** the result to the Serial Monitor for observation.

Understanding this loop equips you to handle many other analog devices—photo resistors, temperature probes, force sensors, etc.—by simply swapping the sensor and adjusting the scaling math.

Important Points

- **ADC resolution:** 10-bit! $2^{10} = 1024$ steps; each step is 4.9 mV when using a 5 V reference.
- **Reference voltage:** By default the Arduino uses the operating voltage (5 V or 3.3 V). Changing the reference (e.g., `analogReference(INTERNAL)`) changes the voltage-to-count relationship.
- **Potentiometer wiring:**
 - Pin 1 to 5 V (or 3.3 V)

- Pin /2 (wiper) !' `A0` (or any analog pin)
- Pin /3 !' GND
- `analogRead(pin)` takes ~100 μ s; reading many channels quickly may require timing considerations.
- `map(value, fromLow, fromHigh, toLow, toHigh)` performs integer linear mapping; for floating point scaling use arithmetic (float).
- `Serial speed: Serial.begin(9600);` sets the baud rate; match it in the Serial Monitor.
- `Noise reduction:` Adding a small capacitor (0.1 μ F) across the wiper and GND can smooth rapid fluctuations.

Examples

1. Basic Potentiometer Reader

```

`cpp
// Arduino sketch: read a potentiometer and print raw & percent values
const int potPin = A0;      // analog input pin
void setup() {
  Serial.begin(9600);      // start serial communication
}
void loop() {
  int raw = analogRead(potPin); // 0 1023
  int percent = map(raw, 0, 1023, 0, 100); // 0 100%
  Serial.print("Raw: "); Serial.print(raw);
  Serial.print("\tPercent: "); Serial.println(percent);
  delay(200);              // small pause for readability
}
`

```

Explanation: The sketch reads the voltage on `A0`, converts the 10 bit count to a percentage, and prints both numbers every 200 /ms. The `map()` function handles the linear conversion.

2. Controlling an LED with the Potentiometer

```

`cpp
const int potPin = A0;
const int ledPin = 9;      // PWM capable pin
void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}
void loop() {
  int raw = analogRead(potPin);      // 0 1023
  int brightness = map(raw, 0, 1023, 0, 255); // PWM range
  analogWrite(ledPin, brightness);    // set LED brightness
  Serial.print("Pot: "); Serial.print(raw);
}
`

```

```

Serial.print(" LED PWM: "); Serial.println(brightness);
delay(100);
}
...

```

Explanation: The potentiometer value is mapped to the 0 255 PWM range used by `analogWrite()`. Turning the knob smoothly changes the LED brightness, demonstrating a practical use of analog input.

Quick Review

1. **What does `analogRead(A0)` return on a 5 /V Arduino?**
2. **How many discrete steps does a 10 bit ADC provide?**
3. **Write the formula (or `map()` call) to convert a raw reading of 0 1023 into a voltage value (0 5 /V).**
4. **Why might you add a capacitor across the potentiometer wiper and ground?**
5. **If you wanted the potentiometer to control a motor speed (0 255 PWM), which Arduino function would you use after mapping?**

Further Reading

- **ADC Reference Options** – `analogReference()` and using external reference voltages.
- **Noise & Filtering** – software averaging, median filters, and hardware low pass filters.
- **Other Analog Sensors** – photoresistors (LDR), thermistors, force sensing resistors (FSR).
- **Resolution vs. Sampling Rate** – trade offs when using faster ADC reads.
- **Using `float` for precise scaling** – when integer `map()` is insufficient.

Happy experimenting! Ø=þ€

